

應用程式開發指南：模組化史都華平台設計與模擬工具

版本：1.10 (彙整版)

日期：2025年8月30日

目標：為遊樂設備行業的技術人員，打造一款基於 Python 的視窗版桌面應用程式，用於史都華平台 (Stewart Platform) (六自由度與三自由度) 的參數化設計、性能分析、動態模擬與驗證。

[開發者註記：版本變更日誌 (累積所有版本)]

(本變更日誌已整合所有從 v1.6 到 v1.10 的變更記錄。舊版變更未被刪除，除非與新版衝突；所有變更均保留完整描述，以確保歷史追蹤性。)

- [從 v1.5 -> v1.6 正式版]
 - [核心修正] 狀態管理與初始化：徹底重構了 `MainWindow`、`SixDofWidget` 及 `ThreeDofWidget` 之間的狀態管理與訊號傳遞機制，解決了在「新建專案」後因狀態污染而導致計算失敗的根本性問題。
 - [核心修正] 方位與自由度標準化：在第四章中，新增了平台標準方位示意圖，並以此為基準，明確定義了所有平移與旋轉自由度 (Roll, Pitch, Yaw) 的運動方向。此標準化是修正所有姿態控制問題的根本依據。
 - [核心修正] 物理模型校正：在第四章中，徹底修正了工作空間分析的後端計算邏輯，確保「可用」與「機械極限」兩種模式使用正確的物理行程範圍。同時，修正了三自由度平台的旋轉運動學模型，確保姿態變化圍繞其幾何中心進行。
 - [核心修正] 控制邏輯校正：在第四章的自由度定義中，明確了 Pitch/Roll 的直覺操作方向，並以此為依據修正了 `main_window.py` 中的前端控制邏輯，確保 +Pitch=抬頭、+Roll=右側下壓。
 - [修改] 核心定義精確化：在第三章中，根據物理現實修正了自然平衡零位高度 H 的計算基準，明確納入 `s_buffer` 的影響。同時補充了加速度單位在 GUI (g) 與核心引擎 (m/s^2) 之間的轉換約定。
 - [修改] 模型註解更新：在第四章中，補充了關於轉動慣量張量 ($\mathbf{I}$) 在逆動力學計算中採用固定近似值的說明，以明確當前模型的計算限制。
 - [新增] UI 結構文件同步：在第五章中，根據程式碼現狀，補充了主視窗中包含一個為未來「驅動系統」功能預留的占位符頁籤的說明。
- [從 v1.6 -> v1.7 穩定版]
 - [核心重構] 四層架構導入：為根除 v1.6 中存在的狀態管理混亂與多執行緒不穩定問題，v1.7 引入了全新的四層應用程式架構，對專案進行了根本性的重構，以實現職責分離與高內聚低耦合。
 - [核心新增] 狀態管理層 (`StateManager`)：新增 `StateManager` 類別，作為應用程式唯一、權威的數據中心 (Single Source of Truth)，集中管理所有持久化狀態。
 - [核心新增] 高層分析層 (`AnalysisEngine`)：新增 `AnalysisEngine` 類別，作為一個獨立的、執行緒安全的背景工作者，專門負責執行耗時的分析任務。
 - [核心新增] 全域最大靜載分析功能：基於 `AnalysisEngine`，實作了由 Grok (xAI) 文件指導的、完全確定性的「全域最大靜載分析」演算法，採用 LHS+SQP 混合策略，確保了分析的全面性與準確性。
 - [核心修正] 多執行緒穩定性：徹底修復了先前版本中因跨執行緒物件引用與生命週期管理不當而導致的 `QThread` 崩潰問題。採用了安全的「任務資料包」模式進行資料傳遞。

- **[核心修正] 單位換算流程**：修復了 **CoreEngine** 與 **AnalysisEngine** 之間因「雙重換算」導致的角度單位（弧度/角度）混亂問題，確保了分析演算法能在正確的物理邊界內進行採樣。
 - **[修改] 職責簡化**：**MainWindow** 的職責被大幅簡化，回歸其作為 UI 總管與事件協調器的核心角色。**CoreEngine** 的職責也更聚焦於純粹的運動學求解。
 - **[新增] 開發歷程記錄**：在第六章中，詳細記錄了 v1.7 開發分支從遇到多執行緒崩潰，到歷經數次除錯與重構，最終達成穩定狀態的全過程，為未來開發提供寶貴經驗。
- **[從 v1.7 -> v1.8 穩定版]**
 - **[核心功能升級] 全域分析模組**：將原有的「全域最大靜載分析」功能，升級為一個可由使用者自行選擇分析模式的「全域最大出力分析」模組。
 - **[UI 修改] 動力學分析面板**：在 **dynamics_widget.py** 中，將原按鈕更名為「分析全域最大出力」，並新增了一個「僅考慮靜態負載 (忽略加速度)」的核取方塊，賦予使用者在靜態與動態分析之間自由選擇的能力。
 - **[後端修改] 分析引擎**：**analysis_engine.py** 已被修改，現在能夠接收來自前端的模式選擇旗標，並根據該旗標決定在計算時是強制將加速度歸零（靜態分析），還是採用使用者在 UI 上設定的實際加速度值（動態分析）。
 - **[核心修正] 執行緒生命週期**：在 v1.8 的測試階段，發現並修正了一個典型的 **QThread** 生命週期管理錯誤。透過在 **on_analysis_finished** 中手動呼叫 **quit()**，確保了背景執行緒在完成任務後能夠被正確地終止和清理，解決了無法連續執行第二次分析的問題。
 - **[新增] 開發歷程記錄**：在第六章中，新增了 v1.8 開發與除錯的完整記錄。
 - **[從 v1.8 -> v1.9 穩定版]**
 - **[核心功能新增] 可選式關節角度限制分析**：為模擬工具新增了一項關鍵功能。使用者現在可以在 UI 介面自由選擇是否在工作空間計算中，考慮平台的基座與活動平台萬向節的最大物理擺動角度限制，使模擬結果更貼近物理現實。
 - **[UI 修改] 幾何參數面板**：在 **geometry_widget.py** 和 **three_dof_widget.py** 中，於「通用與電動缸參數」區塊新增了一個「啟用關節角度限制」的核取方塊。同時，為「基座關節」和「平台關節」的最大擺角輸入列，配備了包含工程建議值的下拉選單，以方便使用者快速設定。
 - **[後端修改] 運動學核心**：**kinematics.py** 中的 **is_pose_valid** 函式與工作空間分析演算法已升級。當角度限制被啟用時，後端會將其作為一個必要的約束條件進行求解。約束的數學模型也從原先不穩定的 **arccos** 形式，升級為更穩健的 **cos(angle) >= cos(limit)** 內積形式。
 - **[核心修正] 工作空間分析穩定性**：為了解決在啟用嚴苛角度限制後，因求解器起始點 (**neutral_pose**) 本身是一個無效點而導致分析必定失敗的數值穩定性問題，在 **kinematics.py** 中引入了一個名為 **_find_feasible_initial_pose** 的預處理步驟。此步驟會先執行一次最佳化，其唯一目標是尋找一個能同時滿足所有行程與角度限制的有效姿態。主分析流程將以這個「可行起始點」為基礎進行搜索，從而徹底根除了分析失敗的問題。
 - **[新增] 開發歷程記錄**：在第六章中，新增了 v1.9 開發與除錯的完整記錄，詳細描述了上述數值穩定性問題的診斷與解決過程。
 - **[從 v1.9 -> v1.10 穩定版]**
 - **[核心模型修正] 引入關節安裝方式選項**：基於關鍵的工程實務洞見，修正了活動平台的物理模型。現在，使用者可以在 UI 介面選擇活動平台關節的安裝方式為「上置式 (法線朝上)」或「下置式 (法線朝下)」，其中「下置式」已被設為預設值，以符合絕大多數真實世界的平台結構。此修正從根本上解決了先前版本中因模型與物理現實不符而導致的角度計算錯誤與求解器收斂失敗問題。

- **[UI術語統一] 全域術語更新**：為了提升清晰度與專業性，對整個應用程式 UI 介面中的平台命名進行了統一。原有的「下平台/上平台」已被全面更新為「**固定平台/活動平台**」。此變更涵蓋了所有群組標題、參數標籤及 3D 視圖中的註記。
- **[UI一致性修正] 補全三自由度診斷介面**：為三自由度平台的「平台計算與分析」區塊，補上了先前版本中缺失的「**零位姿態所需固定平台擺角**」與「**零位姿態所需活動平台擺角**」兩個診斷資訊欄位，確保了兩種平台類型具備一致的功能與資訊回饋。
- **[視覺化微調] 3D 視圖體驗優化**：
 - **中心點標示**：將 **Mobile Center** 更新為 **Motion Center**；縮小了中心點標記的尺寸 (**STYLE_CENTER_POINT_SIZE = 5**)；並分離了標記點與文字標籤，將文字進行偏移顯示，徹底解決了座標值被遮擋的問題。
 - **法線粗細**：透過改用 **pyvista.Tube** 進行繪製，精確地將法線箭頭的粗細調整到與連桿線相匹配的視覺效果。
- **[新增] 開發歷程記錄**：在第六章中，**新增**了 v1.10 開發與除錯的完整記錄，詳細描述了從發現 **145°** 異常擺角，到最終根據工程實務修正物理模型的整個關鍵決策過程。

第一章：專案初始化與環境設定

本章節指導如何在開發電腦上建立專案的初始架構與環境。

1.1 專案目錄架構

[開發者註記：此處的目錄架構已根據最新的 GUI 模組化重構與動力學模組的引入進行更新，移除了已廢棄的 `parameter_widget.py` (理由：v1.6 後該檔案已被模組化替換，無需保留舊結構，以避免混淆；新版中標示為 v1.7 最新架構)]

建議在 `D:\Python_Programs\Stewart_Platform` 路徑下建立以下目錄結構，以反映 v1.7 的最新架構：

```
D:\PYTHON_PROGRAMS\STEWART_PLATFORM\  
├── .venv/  
├── data/  
│   └── projects/  
├── src/  
│   ├── core/  
│   │   ├── __init__.py  
│   │   ├── config.py  
│   │   ├── analysis_engine.py      <-- [新增] 高層分析層  
│   │   ├── dynamics_engine.py  
│   │   ├── kinematics.py  
│   │   └── state_manager.py        <-- [新增] 狀態管理層  
│   └── gui/  
│       ├── __init__.py  
│       ├── controls/  
│       │   ├── __init__.py  
│       │   ├── analysis_widget.py  
│       │   ├── dynamics_widget.py  
│       │   ├── master_geometry_widget.py  
│       │   ├── six_dof_widget.py  
│       │   └── three_dof_widget.py
```

```
|       |— geometry_widget.py  
|       |— main_window.py  
|       |— pyvista_widget.py  
|— tests/  
|— .gitignore  
|— main.py  
|— requirements.txt
```

1.2 虛擬環境建置

使用虛擬環境可以隔離專案依賴，避免與系統全域套件衝突。

1. **開啟終端**: 在 VS Code 中，點擊 **Terminal > New Terminal**。
2. **確認路徑**: 確保終端機的路徑在 **D:\Python_Programs\Stewart_Platform**。
3. **建立虛擬環境**: 執行以下指令，建立名為 **.venv** 的虛擬環境。

```
python -m venv .venv
```

4. **啟動虛擬環境**:

```
.\.venv\Scripts\activate
```

啟動成功後，終端機提示符前會出現 **(.venv)** 字樣。

5. **(可選) 選擇直譯器**: VS Code 可能會自動偵測到虛擬環境。如果沒有，手動點擊右下角的 Python 版本，選擇 **.venv** 中的 Python 直譯器。

1.3 前置套件安裝

1. 在專案根目錄下建立 **requirements.txt** 檔案，並填入以下內容：

```
numpy  
scipy  
PyQt6  
pybullet  
reportlab  
pyvista  
pyvistaqt
```

2. 在已啟動虛擬環境的終端機中，執行以下指令來安裝所有必要的套件：

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

第二章：核心理念與技術棧

2.1 核心理念

- **[修改] 模組化與職責分離: (v1.7 核心升級)** 應用程式採用嚴格的四層架構，將狀態管理、核心計算、高層分析與使用者介面徹底解耦，確保了程式碼的高內聚、低耦合，便於獨立測試、維護與未來擴展。
- **引導式計算:** 程式應作為設計夥伴，能理解參數間的依賴關係，在資料不完整時引導使用者補全，並自動推算可衍生的參數。
- **設計驗證:** 內建驗證機制，主動檢查幾何矛盾與物理極限，確保設計的嚴謹性與安全性。
- **視覺化回饋:** 提供即時的 3D 視覺化回饋，讓設計過程更直觀。
- **中央化設定:** 將所有可調整的預設值、樣式和演算法參數集中到單一的設定檔 (`config.py`) 中，實現設定與程式碼的分離，便於快速微調與維護。
- **使用者狀態追蹤:** 應用程式應能追蹤使用者的修改狀態，並在關閉前主動提示儲存未保存的變更，確保工作不會意外遺失。

2.2 建議技術棧

- 程式語言: Python 3.x
- 數值/科學計算: NumPy, SciPy
- GUI 框架: PyQt6
- 3D 視覺化:
 - **PyVista / PyVistaQT:** (現行方案) 基於 VTK 的高階 API，已在本專案中用於在 PyQt 應用程式中嵌入高效能、互動式的 3D 場景，負責平台的幾何結構與姿態顯示。
- 資料庫: SQLite (用於內建組件資料庫)
- 報告生成: ReportLab / FPDF (用於匯出PDF報告)

第三章：變數與參數定義

本章定義了應用程式中使用的所有關鍵變數，作為唯一的參考標準。

- **GUI 與核心引擎單位約定:**
 - **GUI 層:** 為了符合使用者習慣，所有在圖形介面 (`GeometryWidget`, `DynamicsWidget`) 的輸入框中，長度單位皆為 **毫米 (mm)**。
 - **核心引擎層 (`CoreEngine`):** 為了符合科學計算標準，所有內部計算與參數儲存，長度單位皆為 **米 (m)**。
 - **資料流:** GUI 層在將參數傳遞給核心引擎前，負責將 **mm** 轉換為 **m**。核心引擎回傳計算結果時，GUI 層負責將 **m** 轉換回 **mm** 進行顯示。
 - **[新增] 加速度單位:** GUI 層為了方便使用者輸入，線性加速度單位使用**重力加速度 (g)**。在將參數傳遞給核心引擎前，GUI 層負責將 **g** 值乘以 **9.81** 轉換為核心引擎所使用的 **m/s²** 單位。
- **幾何參數:**
 - R_a (m): 固定平台節點圓的半徑。
 - R_b (m): 活動平台節點圓的半徑。
 - H (m): 自然平衡零位高度 (**Natural Balance Zero Position Height**)。此為一個計算結果，定義為當所有電動缸長度均處於其可用行程中點 ($L + s_{\text{buffer}}/2 + s/2$) 時，平台在保持水平姿態下的高度。它是所有姿態控制和工作空間分析的操作基準零點。
 - h (m): 平台初始高度 (**Initial Height**)。此為一個理論上的物理量，定義為當所有電動缸長度均處於其**最小機械長度** (L) 時，平台在保持水平姿態下的高度。它代表了平台的**物理下限**，對應總

機械行程的起點。

◦ 開發者註記：**H** 與 **h** 的區別

H 是軟體中所有相對運動的參考原點，旨在最大化和對稱化可用工作空間。而 **h** 則代表平台的物理最低點。兩者在概念上和數值上均不相同，請務必加以區分。

- 自然零位偏移 (T_{x0}, T_{y0}) (m): 與 H 同時解算出的，在自然平衡零位下的水平面偏移量。
- 六自由度平台:
 - D_f (m): 固定平台相鄰節點間的較大弦長。
 - d_f (m): 固定平台相鄰節點間的較小弦長。
 - D_m (m): 活動平台相鄰節點間的較大弦長。
 - d_m (m): 活動平台相鄰節點間的較小弦長。
- 相位角 $\Delta\theta$ (rad 或 deg): 相位角 (Phase Angle)，亦可稱為 交錯角 (Stagger Angle) 或 扭轉角 (Twist Angle)。
 - 定義: 定義為活動平台與固定平台之間的相對旋轉角。它是六自由度平台結構的關鍵參數，用於決定上、下節點的相對定位。
 - 計算公式: 相位角 $\Delta\theta$ 可由弦長參數 D_f, d_f, D_m, d_m 推導得出，但為了符合使用者習慣，在 UI 介面中，我們選擇讓使用者直接輸入 $\Delta\theta$ ，並由程式自動計算出相應的弦長值。
 - 物理意義: 相位角控制了平台的穩定性與運動範圍。較大的相位角可提升平台的抗傾覆能力，但可能限制其旋轉自由度。
 - 單位: 在 GUI 中顯示為度數 (deg)，但核心引擎中統一轉換為弧度 (rad) 進行計算。
- 三自由度平台:
 - D (m): 固定平台相鄰節點間的弦長。
 - d (m): 活動平台相鄰節點間的弦長。
- 電動缸參數:
 - L (m): 電動缸的最小機械長度 (Minimum Mechanical Length)。
 - s_s (m): 電動缸的最大可用行程 (Maximum Usable Stroke)。
 - s_{buffer} (m): 電動缸的安全緩衝區 (Safety Buffer)，用於防止電動缸過度伸展或壓縮。
- 動力學參數:
 - m (kg): 平台的總質量 (包括負載)。
 - $\mathbf{I}$ ($\text{kg}\cdot\text{m}^2$): 平台的轉動慣量張量 (Inertia Tensor)。
 - $\mathbf{a}$ (m/s^2): 平台的線性加速度向量。
 - $\mathbf{\alpha}$ (rad/s^2): 平台的角加速度向量。
 - g (m/s^2): 重力加速度，常數 $9.81 \text{ m}/\text{s}^2$ 。
- 姿態參數:
 - $\mathbf{T} = (T_x, T_y, T_z)$ (m): 平台的平移位移向量，相對於自然平衡零位。
 - $\mathbf{R} = (\phi, \theta, \psi)$ (rad 或 deg): 平台的旋轉歐拉角 (Roll, Pitch, Yaw)。
- 計算結果:
 - $\mathbf{L}_i$ (m): 各電動缸的即時長度。
 - $\mathbf{F}_i$ (N): 各電動缸的即時出力 (從逆動力學計算得出)。
 - F_{max} (N): 全域最大出力 (從全域分析得出)。

第四章：數學模型與演算法

[開發者註記：本章從 v1.6 開始新增平台標準方位示意圖和工作空間分析邏輯修正，未刪除舊內容；v1.6 後新增轉動慣量張量說明，未刪除原有模型描述。]

4.1 平台標準方位與自由度定義

(v1.6 新增：以下為平台標準方位示意圖，用於統一所有計算與顯示的方向。)

[假想示意圖描述：固定平台在底部，活動平台在上部，X 軸指向前方，Y 軸指向右方，Z 軸向上。]

- 平移自由度:
 - T_x (Surge): 前後平移 (+ 前進)。
 - T_y (Sway): 左右平移 (+ 右移)。
 - T_z (Heave): 上下平移 (+ 上升)。
- 旋轉自由度:
 - ϕ (Roll): 左右傾斜 (+ 右側下壓)。
 - θ (Pitch): 前後傾斜 (+ 抬頭)。
 - ψ (Yaw): 左右旋轉 (+ 順時針)。

4.2 運動學模型

- 逆向運動學 (Inverse Kinematics): 給定平台的姿態 $\mathbf{P} = (\mathbf{T}, \mathbf{R})$ ，計算各電動缸的長度 L_i 。

公式： $L_i = \|\mathbf{p}_i - \mathbf{q}_i\|$ ，其中 $\mathbf{p}_i$ 為活動平台節點， $\mathbf{q}_i$ 為固定平台節點。
- 工作空間分析:

(v1.6 修正：確保「可用」模式使用 $L + s_{\text{buffer}}/2$ 到 $L + s_{\text{buffer}}/2 + s$ 的範圍；「機械極限」使用 L 到 $L + s + s_{\text{buffer}}$ 。)

使用 SLSQP 最佳化求解邊界。

4.3 動力學模型

- 逆動力學 (Inverse Dynamics): 給定平台的運動參數，計算各電動缸的出力 F_i 。

使用雅可比矩陣 $\mathbf{J}$ ： $\boldsymbol{\tau} = \mathbf{J}^T \mathbf{F}$ ，其中 $\boldsymbol{\tau}$ 為扭矩向量。

(v1.6 新增註解：轉動慣量張量 $\mathbf{I}$ 目前採用固定近似值；未來可擴展為動態計算。)

4.4 全域分析演算法

- LHS + SQP 混合策略: 使用拉丁超立方採樣 (LHS) 生成初始點，然後用序列二次規劃 (SQP) 尋找極值。

(v1.7 新增：用於全域最大靜載分析；v1.8 升級為雙模式 (靜態/動態)。

(v1.9 新增：整合可選關節角度限制，使用內積形式計算。)

第五章：應用程式架構與模組設計

[開發者註記：本章從 v1.6 開始同步 UI 結構說明；v1.7 引入四層架構，未刪除舊模組描述；v1.8/v1.9/v1.10 修改特定職責，未刪除原有內容。]

5.1 層次一：狀態管理層 (state_manager.py)

- 角色: `StateManager` 類別，是應用程式唯一、權威的數據中心 (Single Source of Truth)。
- 職責:

1. **集中管理**: 負責儲存所有需要跨模組共享或需要持久化的應用程式狀態，如專案路徑、工作空間分析結果（可用與機械）、當前平台類型、`is_dirty` 標記等。
2. **狀態接口**: 提供統一的 `get/set` 接口供其他模組讀取或更新狀態，避免狀態數據散落在各個 UI 元件中。
3. **生命週期**: 其生命週期與主視窗相同，確保在整個應用程式運行期間，狀態都是一致且可追溯的。

5.2 層次二：核心計算層

• 1. 運動學核心 (`kinematics.py`)

- 角色: `CoreEngine` 類別，專注於純粹的運動學與幾何求解。
- 職責:
 1. **無狀態計算**: 作為一個計算工具，它本身不儲存高層次的應用程式狀態（如工作空間結果）。它接收基礎參數，執行計算，然後立即回傳結果。
 2. **運動學求解器**: 封裝所有核心數學運算，包括逆向運動學、幾何求解、以及基於 `SLSQP` 最佳化的工作空間邊界分析。
 3. **[修改] 姿態驗證**: 提供 `is_pose_valid` 接口，用於快速檢查一個給定的姿態是否在指定的行程限制內物理可達。此函式現在能夠根據使用者選擇，額外考慮平台的萬向節角度限制，並能根據選擇的「關節安裝方式」採用正確的物理模型進行驗證。
 4. **檔案處理**: 負責將專案的所有參數序列化至 JSON 檔案或從中讀取。

• 2. 動力學核心 (`dynamics_engine.py`)

- 角色: `DynamicsEngine` 類別，專注於底層的逆動力學計算。
- 職責:
 - 接收平台的瞬時幾何狀態與運動參數（加速度）。
 - 建立平台的雅可比矩陣，並求解出各致動器所需的瞬時出力。

5.3 層次三：高層分析層 (`analysis_engine.py`)

- 角色: `AnalysisEngine` 類別，一個獨立的、執行緒安全的背景工作者。
- 職責:
 1. **執行複雜任務**: 專門負責執行如「全域最大出力分析」這類計算密集、耗時長的分析任務，避免主 UI 執行緒被阻塞。
 2. **LHS+SQP 演算法**: 內部實作了基於拉丁超立方採樣與序列二次規劃的混合智慧演算法，用於在整個工作空間內尋找極值點。
 3. **執行緒安全**: 被設計為一個無狀態的物件。它不直接存取任何主執行緒的物件，而是透過一個「任務資料包」接收工作所需的所有數據，並透過 PyQt 的信號與槽機制 (`pyqtSignal`) 安全地將進度與結果回傳給主視窗。
 4. **雙模式分析**: 能夠根據從 `MainWindow` 傳入的旗標，決定是執行純靜態分析（強制加速度為零），還是執行包含加速度的動態分析。

5.4 層次四：使用者介面與協調層 (GUI)

• 1. 主視窗 (`main_window.py`)

- 角色: `MainWindow` 類別，應用程式的 UI 總管與事件協調中心。
- 職責:

1. **UI 組合**: 建立並管理所有 UI 元件的佈局與顯示。(v1.6 新增：包含一個為未來「驅動系統」功能預留的占位符頁籤。)
2. **事件協調**: 這是其核心職責。它負責接收來自 UI 元件的訊號（如按鈕點擊），然後決定呼叫哪個後端模組來處理該事件。
3. **資料流控制**: 負責在各個層次之間傳遞資料。例如，從 **CoreEngine** 獲取計算結果，一份儲存在 **StateManager**，另一份轉換單位後傳遞給 **AnalysisWidget** 顯示。
4. **執行緒管理**: 負責創建、啟動和安全地清理 **AnalysisEngine** 的背景執行緒。在啟動前，它會從 **StateManager**、**CoreEngine** 以及 **DynamicsWidget** 收集數據，打包成「任務資料包」後再派發給背景執行緒。
5. **姿態控制邏輯**: 確保 Pitch/Roll 的控制方向符合操作定義 (+Pitch=抬頭，+Roll=右側下壓)。

- 2. 各功能面板 (**controls/** 目錄下的元件)

- **角色**: 專用的 UI 元件，負責特定功能的「顯示」與「使用者輸入」。
- **dynamics_widget.py**: (v1.8 升級) 其「全域分析」區塊已被修改。原按鈕更名為「分析全域最大出力」，並新增了一個「僅考慮靜態負載 (忽略加速度)」的核取方塊，讓使用者可以自由選擇分析模式。
- **[修改] geometry_widget.py / three_dof_widget.py**: (v1.10 升級) 在「關節物理限制」區塊，新增了「活動平台關節安裝方式」的選項，允許使用者定義正確的物理模型。同時，所有術語已更新為「固定平台/活動平台」。(v1.9 新增：新增「啟用關節角度限制」核取方塊及下拉選單。)
- **職責**: 每個 Widget (如 **DynamicsWidget**, **AnalysisWidget**) 都像一個儀表板。它們的職責是：接收 **MainWindow** 傳來的數據並顯示；監聽使用者的操作（如拖動滑塊），並將這些操作以 **pyqtSignal** 的形式發射出去，通知 **MainWindow** 進行處理。它們自身不包含複雜的應用程式邏輯。

5.5 模組四：輸出與報告 (未來規劃)

- **描述**: 負責將設計和分析的結果以持久化、標準化的格式匯出。
- **功能**: 提供一個「生成設計報告」按鈕，將專案所有關鍵資訊彙整成一份專業的 **PDF 文件**。

第六章：開發者註記與決策記錄

本章旨在記錄在達成各穩定基線的過程中，所遇到的關鍵挑戰、錯誤分析以及最終的解決方案。這份記錄是未來維護與功能迭代的寶貴資產，旨在避免重蹈覆轍。

(本章已累積所有從 v1.6 到 v1.10 的迭代修正記錄。所有問題均筆筆記錄，並註記是否已解決及解決方案。無刪除舊記錄。)

- 6.1 迭代修正一：物理模型與方位定義的標準化

- **主題**：確保軟體的計算、顯示與物理現實完全一致。
- **遇到的問題與決策記錄**：
 1. **行程定義偏差**：初版對「可用行程」的定義忽略了 **s_buffer**，導致工作空間分析不準確。
 2. **方位定義顛倒**：初版六自由度平台的初始方位與設計示意圖顛倒 180 度，是導致旋轉方向混亂的根源。
 3. **旋轉直覺不符**：在方位修正後，純數學的「右手定則」旋轉方向與模擬器操作員的直覺（如 **+Pitch=抬頭**）不符。

4. **3-DOF 旋轉模型錯誤**：初版三自由度平台的旋轉是圍繞「外接圓圓心」而非「幾何中心」，導致動作異常。

- **最終方案**：

1. **明確物理模型**：在第三章中，以圖文方式明確電動缸行程的物理分佈，並以此為基準，重寫了 `kinematics.py` 中所有相關的計算邏輯（H 高度，工作空間分析）。
2. **強制標準方位**：在 `kinematics.py` 中，透過增加 180 度旋轉偏移，強制使六自由度平台的初始方位與第四章新增的「標準方位示意圖」對齊。
3. **前端體驗適配**：在 `main_window.py` 的 `update_3d_view` 函式中，對從 UI 獲取的 Roll 值乘以 `-1`，將純粹的數學旋轉轉換為符合操作直覺的視覺回饋，實現了前後端邏輯的解耦。
4. **修正 3-DOF 模型**：在 `kinematics.py` 中，重寫了三自由度平台的運動學計算函式，確保其旋轉是圍繞幾何中心（形心）進行的。

- **[註記：已解決]** 所有上述問題已在 v1.6 穩定版中透過物理模型校正與方位標準化解決。

- **6.2 迭代修正二：狀態管理與初始化流程的根本性修復**

- **主題**：解決在點擊「新建專案」後，程式無法再次進行計算的嚴重迴歸錯誤。

- **遇到的問題與決策記錄**：

1. **遇到的問題**：經過多次測試，發現問題根源在於應用程式的狀態管理混亂。點擊「新建專案」的流程，與使用者手動修改參數的流程之間存在邏輯衝突，導致程式進入一個不可預測的、被污染的狀態。
2. **錯誤的根源 1 (職責不清)**：子元件 (`six_dof_widget.py`) 在參數被修改時，錯誤地承擔了「禁用按鈕」的職責，導致主視窗 (`main_window.py`) 無法在正確的時機重新啟用它們。
3. **錯誤的根源 2 (重置不全)**：「新建專案」的流程沒有徹底重置所有相關的 UI 元件（特別是 `AnalysisWidget`），導致舊的狀態殘留。
4. **錯誤的根源 3 (訊號鏈路中斷)**：在模組化重構過程中，按鈕的 `clicked` 訊號未能正確地從子元件傳遞到主視窗。

- **最終方案：確立單一、權威的狀態管理流程**：

1. **`on_new_project` 為唯一重置入口**：`main_window.py` 中的 `on_new_project` 被確立為唯一、權威的應用程式重置函式。任何需要重置專案的操作（點擊「新建專案」按鈕、切換平台類型）都必須呼叫此函式。此函式負責徹底重置所有後端引擎與所有相關 UI 元件（包括幾何參數面板和姿態控制面板）。
2. **`parameter_changed` 職責簡化**：子元件中的 `_on_parameter_changed` 函式被重構。其唯一職責是發射 `parameter_changed` 訊號。
3. **`_mark_as_dirty` 職責釐清**：`main_window.py` 中的 `_mark_as_dirty` 函式現在是處理 `parameter_changed` 訊號的唯一插槽。其唯一職責是將 `is_dirty` 旗標設為 `True`，並呼叫子元件的 `reset_all_calculated_values()` 來清除過期的計算結果並禁用後續按鈕。
4. **`on_parameters_confirmed` 職責**：此函式在執行任何新計算之前，會先呼叫 `core_engine.reset_calculated_parameters()`，確保後端處於絕對乾淨的狀態，從而保證了計算的可靠性。

- **[註記：已解決]** 所有上述問題已在 v1.6 穩定版中透過狀態管理重構解決。

- **6.3 渲染策略的演進**

- **主題**：解決 3D 動畫過程中的背景閃爍問題。

- **遇到的問題與決策記錄**：

1. **初期問題**：在動畫的每一幀都呼叫 `plotter.clear()`，導致靜態的座標軸背景不斷閃爍，視覺體驗不佳。

- **最終方案：**採用「原地更新 (Update-in-Place)」策略。重量級的 `rebuild_scene()` 函式只在需要徹底刷新時（如讀取專案）呼叫，它負責建立所有演員 (Actor)。而輕量級的 `update_platform_pose()` 函式則在動畫和滑塊拖動時高頻呼叫，它絕不清除場景，而是透過 `actor.mapper.dataset.copy_from()` 這個高效的 API，只更新模型的頂點數據，從而根除了閃爍問題。
- **[註記：已解決]** 此問題已在 v1.6 穩定版中透過渲染策略優化解決。

• 6.4 3D 視窗 UI 整合的挑戰

- **主題：**為 3D 視窗添加浮動的視角控制按鈕。
- **遇到的問題與決策記錄：**
 1. **已棄用的方案：**曾嘗試將一個純 Qt 元件 (`ViewControlsWidget`) 作為 `PyVistaWidget` (`QtInteractor`) 的子元件。
 2. **遇到的問題：**該方案導致了 Qt 和 VTK 兩個渲染引擎之間的嚴重衝突，具體表現為：終端機出現 `QWidget::paintEngine: Should no longer be called` 警告；作為子元件的按鈕面板被 VTK 渲染層完全覆蓋而無法顯示；`PyVistaWidget` 內部的事件處理循環被擾亂，導致互動式座標軸標記 (Gizmo) 無法響應點擊。
- **最終方案：**採用「徹底解耦」的策略。將所有視角控制按鈕移至主視窗 (`MainWindow`) 中進行管理。`MainWindow` 作為一個純 Qt 環境，可以穩定地建立和顯示按鈕。當按鈕被點擊時，`MainWindow` 再向 `PyVistaWidget` 發出單向的、清晰的指令來改變視角。這個方案徹底避免了渲染衝突，並使得各模組的職責更加單一和清晰。
- **[註記：已解決]** 此問題已在 v1.6 穩定版中透過 UI 解耦解決。

• 6.5 API 使用的穩健性

- **主題：**確保程式在不同版本的 PyVista 函式庫下都能穩定運行。
- **遇到的問題與決策記錄：**
 1. **遇到的問題：**初期程式碼直接存取了 `camera.view_up` 或 `actor.copy_from` 等底層或特定版本的 API，導致了在某些環境下出現 `AttributeError` 和 `ValueError`。使用了在舊版本中不存在的 `remove_actor(..., on_missing='ignore')` 參數，導致了 `TypeError`。
- **最終方案：**
 1. 轉而使用 PyVista 提供的高階、穩健的 API。例如，使用 `plotter.camera_position` 來一次性處理視角的方位，使用 `actor.mapper.dataset` 來更新模型數據。
 2. 放棄對特定版本功能的依賴，改用更具普適性的「手動檢查」邏輯（在移除演員前，先檢查其是否存在），確保了程式的向後相容性。
- **[註記：已解決]** 此問題已在 v1.6 穩定版中透過 API 穩健化解決。

• 6.6 迭代修正三：v1.7 多執行緒穩定性重構

- **主題：**解決在點擊「分析全域最大靜載」後，程式因多執行緒問題而卡死或崩潰的根本性錯誤。
- **遇到的問題與決策記錄：**
 1. **初步診斷 (`AttributeError`)：**初次崩潰來自於 `MainWindow` 呼叫了 `CoreEngine` 中一個不存在的方法 (`reset_calculated_parameters`)。雖然補上該方法能解決直接崩潰，但這揭示了更深層次的狀態管理問題：誰應該在何時重置何種狀態？
 2. **二次診斷 (單位換算錯誤)：**解決 `AttributeError` 後，分析依然失敗，並提示「未能找到任何有效姿態點」。經追蹤發現，`CoreEngine` 在回傳工作空間時錯誤地將弧度轉為角度，而 `AnalysisEngine` 又再次進行了轉換，導致採樣範圍被放大了數千倍，所有採樣點均無效。

3. **最終診斷 (QThread Destroyed)**：修正單位問題後，分析終於可以運行，但在完成的瞬間，程式以 `QThread: Destroyed while thread is still running` 的錯誤崩潰。根本原因在於 `MainWindow` 的物件生命週期管理不當：它在收到 `analysis_finished` 訊號後，過早地將 `self.analysis_thread` 設為 `None`，導致背景執行緒在其標準退出流程完成前就被強制銷毀。

- **最終方案：確立 v1.7 的四層架構與安全執行緒模型：**

1. 引入 **StateManager**：為了解決狀態混亂問題，引入 `StateManager` 作為唯一數據中心。`CoreEngine` 不再儲存工作空間等高層次狀態。
2. **嚴格單位劃分**：確立了「後端統一用弧度，前端協調器負責轉角度」的原則。`CoreEngine` 只回傳純淨的弧度值；`MainWindow` 接收後，一份存入 `StateManager`，另一份複製轉換後再送往 UI 顯示。
3. **執行緒安全重構**：
 - 將 `AnalysisEngine` 改造為完全無狀態的獨立工作者，其 `__init__` 不接收任何主執行緒物件。
 - `MainWindow` 採用「任務資料包」模式，在啟動執行緒前打包好所有數據，透過 `lambda` 函式安全地傳遞給 `AnalysisEngine`。
 - `MainWindow` 採用基於 `finished` 訊號的清理機制，確保只有在 `QThread` 完全結束其生命週期後，才安全地釋放對其物件的引用，徹底根除了崩潰問題。

- **[註記：已解決]** 所有上述問題已在 v1.7 穩定版中透過四層架構與執行緒重構解決。

- **6.7 迭代修正四：v1.8 全域分析功能升級與執行緒生命週期完善**

- **主題**：將「全域最大靜載分析」升級為可選模式的「全域最大出力分析」，並修復在連續執行分析時遇到的 `QThread` 狀態未重置問題。
- **遇到的問題與決策記錄**：
 1. **功能單一性**：v1.7 的全域分析功能只能計算靜態負載，無法評估在包含加速度的動態工况下的全局最差點，限制了其工程應用價值。
 2. **執行緒狀態未重置**：在 v1.8 的開發測試中發現，當一次全域分析成功結束後，`QThread` 物件的 `isRunning()` 狀態依然為 `True`，導致使用者無法啟動第二次分析，而是錯誤地觸發了「中止分析」的對話框。
 3. **根本原因**：問題的根源在於，背景執行緒在完成任務後，其內部的事件迴圈 (Event Loop) 處於閒置狀態，但從未接收到終止指令。`quit()` 這個終止指令被錯誤地連接到了 `finished` 訊號上，但 `finished` 訊號只有在事件迴圈終止後才會發出，造成了死結。

- **最終方案：功能擴充與生命週期閉環：**

1. **UI 擴充**：在 `DynamicsWidget` 中新增一個核取方塊，允許使用者明確選擇分析模式（靜態或動態）。
2. **數據鏈路打通**：`MainWindow` 在打包「任務資料包」時，會讀取該核取方塊的狀態，並將其作為旗標一同傳遞給 `AnalysisEngine`。
3. **後端邏輯適配**：`AnalysisEngine` 內部根據收到的旗標，決定是強制將加速度歸零，還是使用 `task_data` 中包含的實際加速度值進行計算，實現了雙模式分析。
4. **生命週期閉環**：在 `MainWindow` 中，將 `quit()` 的觸發時機，從 `finished` 訊號，改為在 `on_analysis_finished` 槽函式的結尾處手動呼叫。這確保了主視窗在確認收到結果後，會主動、明確地向背景執行緒發出「任務結束，可以下班」的指令，從而使其能夠正常終止、觸發 `finished` 訊號，並最終被安全地清理，為下一次分析做好了準備。

- **[註記：已解決]** 所有上述問題已在 v1.8 穩定版中透過功能升級與執行緒生命週期完善解決。

- **6.8 迭代修正五：v1.9 工作空間分析穩定性與物理約束實作**

- **主題：**實作「可選式關節角度限制」功能，並解決因此引入的、極其頑固的 **SciPy** 最佳化求解器數值穩定性問題。
- **遇到的問題與決策記錄：**
 1. **初始問題：**在 **kinematics.py** 的工作空間分析中加入關節角度作為約束條件後，分析流程必定失敗，回傳一個零範圍的工作空間，且求解器 (**SLSQP**, **trust-constr**) 無法收斂。
 2. **初步嘗試失敗：**嘗試了更換求解器、提供解析梯度 (**jac/hess**)、為起始點引入微擾動等多種方案，均未成功。這證明問題的根源比預想的更為根本。
 3. **[關鍵] 根本原因診斷 (AI 協作)：**引入外部 AI (Grok, ChatGPT, Gemini) 進程式碼審查與理論分析，得到了兩個決定性的洞見：
 - **無效的起始點：**失敗的最根本原因，是我們的求解器起始點 **neutral_pose** (零位姿態)，在嚴苛的角度限制下，其本身就是一個物理上不可能達成的「無效點」。求解器從一個無效點出發，自然無法在約束空間內找到任何有效的探索路徑，從而導致失敗。
 - **不穩健的數學模型：**在約束條件中直接使用 **arccos** 函數來計算角度，雖然在數學上是正確的，但在數值計算上是不穩定的，特別是在接近其定義域邊界 (-1 或 1) 時，其梯度會趨近於無窮大，對求解器造成干擾。
- **最終方案：根本性的演算法重構：**
 1. **升級約束模型：**將所有角度限制的計算，從 **arccos** 形式，升級為更穩定、對求解器更友好的 **cos(angle) >= cos(limit)** 內積 (點積) 比較形式。這避免了 **arccos** 的不穩定區間。
 2. **新增「可行點搜索」預處理：**在正式進行邊界搜索之前，新增了一個名為 **_find_feasible_initial_pose** 的預處理步驟。此步驟會先執行一次最佳化，其唯一目標就是尋找一個能夠同時滿足所有行程與角度限制的有效姿態點。
 3. **安全地引導求解器：**只有在成功找到一個「可行起始點」後，主分析流程才會以這個被驗證過的「安全點」為基礎，開始進行工作空間的邊界搜索。如果連一個這樣的點都找不到，程式現在會向使用者彈出一個清晰的、說明設計衝突的提示視窗，而不是回傳零或崩潰。
- **[註記：已解決]** 所有上述問題已在 v1.9 穩定版中透過演算法重構解決。

• 6.9 迭代修正六：v1.10 物理模型修正與 UI/UX 最終完善

- **主題：**根據工程實務，從根本上修正活動平台的物理模型，並統一全域 UI 術語，解決一系列因此產生的連鎖問題，最終達成一個功能完整且互動邏輯清晰的穩定版本。
- **遇到的問題與決策記錄：**
 1. **[關鍵] 根本原因診斷 (來自使用者的工程洞見)：**在 v1.9.x 的反覆測試中，即使程式邏輯不斷完善，「啟用關節限制後計算零位必定失敗」的問題依然存在。最終，由使用者提出了一個決定性的觀點：「**真實世界的活動平台關節大多為下置式安裝，其法線應朝下計算**」。這一洞見揭示了先前所有失敗的根源：我們的軟體一直採用一個與工程現實不符的「上置式」物理模型，導致計算出的理論擺角 (~145°) 遠超物理極限，求解器回報失敗是完全正確的。
 2. **UI 術語混淆：**使用者回饋，UI 中使用的「基座」與「平台」等術語容易造成混淆，尤其是在指代活動平台時不夠清晰。
 3. **視覺化細節：**使用者回饋 3D 視圖中的中心點標記過大且遮擋座標文字，法線箭頭過粗，影響了視覺體驗。
- **最終方案：確立 v1.10 穩定基線：**
 1. **修正核心物理模型：**在 **kinematics.py** 中，新增了 **platform_joint_style** 參數。**_calculate_joint_angles** 函式現在會根據此參數，決定平台法線的方向 (朝上或朝下)，從而使用正確的物理模型進行角度計算。

2. **升級 UI 功能**：在 `geometry_widget.py` 中，新增了「活動平台關節安裝方式」的單選按鈕，並根據使用者的建議，將符合工程實務的「下置式 (法線朝下)」設為預設選項。
 3. **統一全域術語**：對所有 `gui/` 目錄下的相關檔案進行了一次全面的文字審查與替換，將「下平台/上平台」統一更新為「固定平台/活動平台」，並將 3D 視圖中的 `Mobile Center` 更新為 `Motion Center`。
 4. **完善診斷功能**：為 `three_dof_widget.py` 補上了缺失的「零位姿態所需擺角」的診斷資訊欄位，確保了 UI 的一致性。
 5. **微調視覺效果**：在 `config.py` 中縮小了中心點標記的尺寸 (`STYLE_CENTER_POINT_SIZE = 5`)；在 `pyvista_widget.py` 中分離了中心點的標記與文字，並對文字進行偏移，以解決遮擋問題；同時，改用 `pyvista.Tube` 繪製法線，使其粗細與連桿匹配。
- **[註記：已解決]** 所有上述問題已在 v1.10 穩定版中透過物理模型修正與 UI 統一解決。
-

第七章：推薦的第三方函式庫與資源

1. 3D 視覺化: `PyVista` 與 `PyVistaQT`

- **用途**: 這兩個函式庫是實現**模組 5.3** 中 3D 視覺化介面的核心。`PyVista` 提供了一套對 `VTK` 的高階封裝，簡化了 3D 物件的創建與操作；`PyVistaQT` 則提供了 `QtInteractor` 元件，讓 `PyVista` 渲染視窗可以成為獨立視窗並與主程式互動。
- **資源**: `PyVista` 官方文件 (<https://docs.pyvista.org/>) 及 `PyVistaQT` GitHub 儲存庫。

2. 物理模擬 (未來擴展): `PyBullet`

- **用途**: 未來若要擴展動態模擬、碰撞檢測等功能，`PyBullet` 將會是首選的物理引擎。
- **資源**: `PyBullet` 官方指南 (<https://pybullet.org/wordpress/>)。